Database Tables — Detailed Guide

Updated: 2026-04-17 | Based on actual production DB (5 patients processed)
Total: 10 tables | Branch: `feat/save-intermediate-results`

---

1. `transcript_analysis_log` (5 rows)

Pipeline execution record — stores the fact that "this patient file was processed" along with all metadata about the processing.

When the pipeline processes `Input_Keystrokes REC001 (SID 14).xlsx`, one row is created in this table. It records when processing started (`pipeline_started_at`), how many sentences were analyzed (`total_sentences=424`), what settings were used (`top_n=10`), a backup of the result xlsx (`xlsx_data`), whether the AI pipeline completed (`processed=True`), and the overall consultation quality score (`ai_overall_score=3.67`). Next time the same file is encountered, this table is checked and already-processed files are skipped.

Column	Type	Sample (SID_10)	Description
`id`	SERIAL PK	1	Unique analysis run ID
`patient_id`	VARCHAR	SID_10	Patient identifier (extracted from filename)
`total_sentences`	INT	428	Number of sentences after R stringi segmentation
`top_n`	INT	10	Top-K sentences selected per domain
`context_window`	INT	3	±3 surrounding sentences for context
`model_results`	JSONB	None	(Deprecated — kept for backward compatibility)
`xlsx_data`	BYTEA	(18,300 bytes)	Result xlsx backup. Fallback when disk file deleted
`source_filename`	VARCHAR	Input_Keystrokes REC 001 (SID 10).xlsx	Original input filename
`pipeline_started_at`	TIMESTAMP	05:26:14	When pipeline_runner began processing

Column	Type	Sample (SID_10)	Description
`analyzed_at`	TIMESTAMP	05:26:30	When NLP results saved to DB (Step 8)
`ai_overall_score`	FLOAT	3.4	Average of all domain ai_scores (0-5). Displayed on DOCTOR page
`processed`	BOOLEAN	True	True when full pipeline (NLP + AI) completed
`processed_at`	TIMESTAMP	05:29:28	When AI pipeline (Step 9) completed

Used by: - `persistence.file_already_processed()` — skip already-processed files - `GET /api/transcript/download/{patient_id}` — xlsx download fallback from `xlsx_data` - `GET /api/transcript/history/{patient_id}` — analysis run history - `llm_domain_scoring_and_summary.analysis_id` FK target

---

2. `sentence_prediction` (250 rows = 5 patients × 50 sentences)

NLP model predictions — stores the Top-10 sentences per domain with their probability scores, as determined by the 5 NLP classification models.

For example, out of SID 10's 428 sentences, the cp (cancer prognosis) model assigned 95.1% probability to "so i'm going to take that 12 percent and cut it in half…", making it one of the Top-10 for that domain. 5 domains × 10 sentences = 50 rows per patient. The patient page uses this data to show "your doctor said these things" evidence sentences.

Column	Type	Sample (SID_10, cp)	Description
`id`	SERIAL PK	1	Unique prediction ID
`analysis_id`	INT FK	1	→ transcript_analysis_log.id
`patient_id`	VARCHAR	SID_10	Patient identifier
`model`	VARCHAR	cp	Domain (cp/le/ed/inc/ius)
`sentence_index`	INT	166	Global sentence sequence number

Column	Type	Sample (SID_10, cp)	Description
`utterance_index`	INT	67	Original utterance number (i)
`sentence_in_utterance`	INT	3	Sentence number within utterance (i2)
`speaker`	VARCHAR	Interviewer:	Speaker
`sentence_text`	TEXT	"so i'm going to take that 12 percent and cut it in half…"	Original sentence
`pred_score`	FLOAT	0.951	NLP probability (95.1% related to cp)
`context`	TEXT	"it removes the testosterone…<main>so i'm going to…</main>…"	±3 surrounding sentences

Used by: - `GET /api/patient/sentences/{file}` — evidence sentences on PATIENT page - Analysis result reconstruction from DB

---

3. `doctor_sentence_view` (221 rows)

Doctor dashboard sentence list — stores deduplicated sentences for the doctor to review and optionally rewrite. Unlike sentence_prediction where the same sentence can appear in multiple domains, here each sentence appears only once with one representative domain.

The reason this table has 221 rows (less than sentence_prediction's 250) is deduplication: if the same sentence ranked in Top-10 for both cp and le domains, it appears in sentence_prediction twice but in doctor_sentence_view only once. The doctor sees each sentence exactly once. When the doctor rewrites a sentence, `(file, i, i2)` from this table identifies the target.

Column	Type	Sample	Description
`file`	VARCHAR PK	Input_Keystrokes REC 001 (SID 10).xlsx	Patient file
`i`	INT PK	67	Utterance number
`i2`	INT PK	3	Sentence within utterance
`speaker`	VARCHAR	Interviewer:	Speaker
`sentence`	TEXT	"so i'm going to take that 12 percent…"	Sentence text

Column	Type	Sample	Description
score	FLOAT	None	Quality score (can be populated later)
class	VARCHAR	cancer_prognosis	Representative domain
time	TIMESTAMP	05:26:30	Creation time

Used by: - `GET /api/doctor/sentences/{file}/{speaker}` — DOCTOR page sentence list - `GET /api/doctor/files` — processed patient file list (DISTINCT file) - `PUT /api/doctor/rewrites` — rewrite target identification via (file, i, i2) - `GET /api/patient/sentences/{file}` — evidence sentences on PATIENT page

---

4. `doctor_rewrite_log` (0 rows)

Doctor rewrite history — records every time a doctor modifies a sentence on the dashboard, storing both the original and revised versions with timestamps.

Currently 0 rows because no doctor has rewritten any sentences yet. When a doctor changes "so i'm going to take that 12 percent…" to "Your cancer risk is about 6% with treatment", both versions are INSERT'd here. Multiple rewrites of the same sentence create multiple rows, preserving the full edit history.

Column	Type	Example	Description
file	VARCHAR PK,FK	Input_Keystrokes REC 001 (SID 10).xlsx	Target file
i	INT PK,FK	67	Target utterance number
i2	INT PK,FK	3	Target sentence number
time	TIMESTAMP PK	2026-04-17T14:30:00Z	Rewrite time (multiple rewrites = multiple rows)
speaker	VARCHAR	Interviewer:	Speaker
original_sentence	TEXT	"so i'm going to take…"	Original sentence
revised_sentence	TEXT	"Your cancer risk is about 6%…"	Doctor's rewritten version
score	FLOAT	4.0	Quality score after rewrite
class	VARCHAR	cancer_prognosis	Domain

Used by: - `GET /api/doctor/rewrites` — rewrite history on DOCTOR page - `GET /api/doctor/rewrites/{file}/{i}/{i2}/history` — per-sentence rewrite history

---

5. `patient_summary` (5 rows)

Patient summary parent record — represents "a summary exists for this patient". Acts as the FK parent for patient_summary_domain (5 domain rows per patient).

One row per patient. The actual domain-level data (summary_text, patient_scoring) lives in the child table `patient_summary_domain`. This table exists because DB design requires a "1" side table in a 1:N relationship for FK integrity. Deleting a patient here CASCADE-deletes all 5 domain rows.

Column	Type	Sample	Description
`file`	VARCHAR PK	Input_Keystrokes REC 001 (SID 10).xlsx	Patient file
`speaker`	VARCHAR PK	Patient_Input_Keystrokes REC 001 (SID 10)	Patient identifier
`entire_summary`	TEXT	None	Full visit summary (currently unused)

Used by: - `GET /api/patient/summaries/{file}/{speaker}` — JOINs with patient_summary_domain - FK parent for patient_summary_domain CASCADE delete

---

6. `patient_summary_domain` (25 rows = 5 patients × 5 domains)

Patient per-domain feedback — stores the star rating and text response that the patient enters on the dashboard for each domain. The pipeline creates empty rows (NULL values), and the patient fills them in later during a follow-up visit.

Like a patient survey form: the pipeline creates 5 blank domain cards (cp, le, ed, inc, ius), and when the patient clicks a star rating for "cancer prognosis", `patient_scoring` is UPDATE'd from NULL to the rating value. Note: `patient_response` (free text) has an API but no UI implementation yet.

Column	Type	Sample (SID_10, cp)	Description
`file`	VARCHAR PK,FK	Input_Keystrokes REC 001 (SID 10).xlsx	Patient file
`speaker`	VARCHAR PK,FK	Patient_…SID 10	Patient identifier
`domain`	VARCHAR PK	cancer_prognosis	Domain
`display_order`	INT	1	UI display order (1=cp, 2=inc, 3=ed, 4=ius, 5=le)
`summary_text`	TEXT	“”	Domain summary text (currently empty)
`patient_scoring`	INT	None	Patient star rating 0-10 (NULL until patient rates)
`patient_response`	TEXT	None	Free text feedback (API exists but UI NOT implemented)

Used by: - `GET /api/patient/summaries/{file}/{speaker}` — domain summary cards on PATIENT page - `PUT /api/patient/scoring` — patient star rating (PatientInitialVisitReportV35.tsx) - `PUT /api/patient/responses` — patient text response (API only, UI not implemented)

---

7. `survey_submission_log` (0 rows)

Patient survey responses — stores the full JSON answers when a patient submits SDM, DCS, Risk Perception, or Satisfaction surveys on the follow-up page, and tracks synchronization status with the REDCap research database.

Currently 0 rows because no patient has submitted a survey yet. When a patient completes the DCS (16 questions), the entire response is stored as JSON in `answers`. Immediately after, a background task calls the REDCap API to sync. Same patient submitting the same survey again creates a NEW row (INSERT, not UPDATE) to preserve responses at different timepoints.

Column	Type	Example	Description
`id`	SERIAL PK	1	Submission ID

Column	Type	Example	Description
`file`	VARCHAR FK	Input_Keystrokes REC001 (SID 14).xlsx	Patient file
`speaker`	VARCHAR FK	Patient_…SID 14	Patient identifier
`survey_type`	VARCHAR	dcs	Survey type (sdm/dcs/risk_perception/satisfaction)
`answers`	JSONB	{q1: 2, q2: 3, …, q16: 4}	Full survey response
`extra_data`	JSONB	{browser: Chrome, session: abc}	Metadata
`submitted_at`	TIMESTAMP	2026-04-17T16:05:30Z	Submission time
`redcap_synced`	BOOLEAN	true	REDCap sync success
`redcap_record_id`	VARCHAR	REC-2026-0014	REDCap record ID (on success)
`redcap_error`	TEXT	None	Error message (on failure)

Used by: - `POST /api/surveys/submit` — survey submission from PATIENT follow-up page - `GET /api/surveys/by-speaker/{speaker}` — restore previous responses on page refresh - Background REDCap sync worker

---

8. `llm_domain_scoring_and_summary` (33 rows)

GPT-4o AI pipeline results — stores the per-domain evaluation of "how specifically did the doctor communicate risk to the patient", including a 0-5 score, extracted numerical estimates, and a patient-friendly reformatted summary.

While NLP (sentence_prediction) determines "which sentences are related to this domain", this table stores "how specifically did the doctor communicate risk in those sentences". Example: "There's some cancer risk" = ai_score 1 (vague), "24% risk without treatment, decreases to 6% with surgery" = ai_score 4 (very specific). The 33 rows (more than 5×5=25) occur because some domains have multiple treatment comparisons stored as separate rows.

Column	Type	Sample (SID_10, cp)	Description
`id`	SERIAL PK	1	Result ID

Column	Type	Sample (SID_10, cp)	Description
analysis_id	INT FK	1	→ transcript_analysis_log.id
patient_id	VARCHAR	SID_10	Patient identifier
domain	VARCHAR	cp	Domain
ai_score	INT	4	GPT-4o score 0-5 → **DOCTOR page**
score_explanation	TEXT	"Does the text mention cancer mortality?… score is 4."	GPT-4o chain-of-thought reasoning (from ai_pipeline/scoring.py)
extracted_estimate	TEXT	"24, 25 percent—down to six percent"	Extracted risk numbers
treatment	VARCHAR	None	Related treatment (ed/inc/ius only)
source_sentence	TEXT	"so i'm going to take that 12 percent…"	AI-selected original sentence
source_context	TEXT	"it removes the testosterone…<main>…</main>…"	Surrounding context
reformat_sentence	TEXT	"Your doctor noted that your risk of dying of cancer is 24–25%…"	Patient-friendly text → **PATIENT page**
source_filename	VARCHAR	Input_Keystrokes REC 001 (SID 10).xlsx	Source file
created_at	TIMESTAMP	05:29:28	Creation time

Used by: - `GET /api/doctor/scores/average` — consultation quality score on DOCTOR page - `GET /api/doctor/scores/trajectory` — score trend over time on DOCTOR page - `GET /api/patient/ai-summary/{file}` — AI summary card on PATIENT page - `POST /api/doctor/ai-rewrite` — AI rewrite suggestions using source_sentence + source_context

---

9. `user_interaction_log` (108 rows)

User behavior tracking — records every user action on the dashboard (clicks, scrolls, mouse movements, dwell time) in real-time for research analysis of how patients and doctors interact with the consultation information.

For example, when a patient hovers near the "cancer prognosis" card for 38 seconds, an event is recorded with `event_type=cursor_proximity_leave` and `hoverDuration: 38007071ms`. This data enables research questions like

“Which domain information do patients spend the most time reading?” or “Which sentences do doctors click on most?”

Column	Type	Sample	Description
`id`	SERIAL PK	1	Event ID
`session_id`	VARCHAR	session_1776435900666_syzse2gio6lrk	Browser session ID
`role`	VARCHAR	patient	User role (patient/physician)
`visit_type`	VARCHAR	first	Visit type (first/followup)
`file`	VARCHAR	Input_TurboScribe SID 33.csv	Patient being viewed
`speaker`	VARCHAR	Patient_Input_TurboScribe SID 33	Patient identifier
`event_type`	VARCHAR	cursor_proximity_leave	Event type
`element_id`	VARCHAR	/	Target UI element
`event_data`	JSONB	{cursorX: 0, hoverDuration: 38007071, …}	Event details
`device_type`	VARCHAR	desktop	Device type
`client_timestamp`	TIMESTAMP	14:25:00	Client-side time
`created_at`	TIMESTAMP	14:25:00	Server save time

Used by: - `POST /api/tracking/events` — event batch submission from frontend - `GET /api/tracking/stats` — admin tracking dashboard - `GET /api/tracking/analytics` — behavior analysis

---

10. `session_recording` (2 rows)

Session replay data — stores rrweb-recorded user sessions as binary chunks, enabling video-like playback of how users navigated the dashboard. PHI (Protected Health Information) is masked so patient names and sentence text appear as “*” in recordings.**

While `user_interaction_log` records individual events, this table stores the raw data needed to replay the entire session visually. Chunks are sent every 30 seconds or every 500 events. The admin tracking dashboard can play back these recordings to observe user behavior.

Column	Type	Sample	Description
`id`	SERIAL PK	1	Recording ID
`session_id`	VARCHAR	rec_1776397890521_dcp3d5x	Recording session ID

Column	Type	Sample	Description
`chunk_index`	INT	0	Chunk number (split every 30s or 500 events)
`file`	VARCHAR	Input_TurboScribe SID 33.csv	Patient being viewed
`visit_type`	VARCHAR	first	Visit type
`recording_data`	BYTEA	(87 bytes)	rrweb event data (PHI masked)
`event_count`	INT	2	Events in this chunk
`created_at`	TIMESTAMP	14:25:30	Server save time

Used by: - `POST /api/tracking/recordings` — chunk upload from frontend (sessionRecorder.ts) - `GET /api/tracking/recordings` — recording list on admin dashboard - `GET /api/tracking/recordings/{sessionId}` — session replay playback